Value Iteration Modifications

2024-11-16, 13:06:54

User

def value_iteration(states, actions, transitions, rewards, gamma=gamma, epsilon=epsilon): # 1. Initialize value function and policy for all states V = np.zeros(len(states)) policy = np.zeros(len(states), dtype=int)

```
iteration = 0
history = []  # History for debugging

while True:
delta = 0
iteration_values = V.copy()
# 2. Perform value iteration across all states

for s in states:

action_values = []
best_action = None
best_value = float('-inf')

# 2.1 Evaluate each action
for a in actions:
value = 0
for next_s, prob in transitions(s, a).items():
value += prob * (rewards(s, a, next_s) + gamma * V[next_s])

action_values.append(value)

print(s, a, action_values)

best_value = max(action_values)
best_action = np.argmax(action_values)

# 2.2 Update the value function and policy
delta = max(delta, abs(V[s] - best_value))
V[s] = best_value
policy[s] = best_action

# Update the history (just for debugging)
history.append(iteration_values)
iteration += 1

#3. Check for convergence
if delta < epsilon:
break

return V, policy, history
```

just modify it , so it saves values for each state (when we move to that state - i mean next state)

2024-11-16, 13:06:55

Assistant gpt-4o-mini

To modify the `value_iteration` function such that it saves the values for each state when

we transition to that state, we need to maintain a record of state-action-next state transitions. We can achieve this by introducing a structure (e.g., a dictionary) to store the values for each state when transitioning from one state to another. Below is the updated code:

```
import numpy as np

def value_iteration(states, actions, transitions, rewards, gamma=0.9,
epsilon=1e-5):
# 1. Initialize value function and policy for all states
V = np.zeros(len(states))
policy = np.zeros(len(states), dtype=int)

iteration = 0
history = []  # History for debugging
state_values = {s: [] for s in states}  # Dictionary to store values for
each state

while True:
delta = 0
iteration_values = V.copy()
# 2. Perform value iteration across all states

for s in states:

action_values = []
best_action = None
best_value = float('-inf')

# 2.1 Evaluate each action
for a in actions:
value = 0
for next_s, prob in transitions(s, a).items():
reward_value = rewards(s, a, next_s)
value += prob * (reward_value + gamma * V[next_s])

# Save the value for this transition (s, a, next_s)
state_values[next_s].append(reward_value + gamma * V[next_s])

action_values.append(value)

best_value = max(action_values)
best_action = np.argmax(action_values)

# 2.2 Update the value function and policy
delta = max(delta, abs(V[s] - best_value))
V[s] = best_value
policy[s] = best_action

# Update the history (just for debugging)
history.append(iteration_values)
iteration += 1

# 3. Check for convergence
if delta < epsilon:
break

return V, policy, history, state_values  # Return state_values with the
result

# Example usage:
# states = [...], actions = [...], transitions = <function>, rewards =
<function>
# V, policy, history, state_values = value_iteration(states, actions,
transitions, rewards)
```

Key Changes Made:

1. **Added `state_values` Dictionary:**
2.

This dictionary, initialized to store lists for each state, keeps track of the values when transitioning to that state.

3.

Saving Values:

4.

Inside the loop for evaluating actions, the calculated value is appended to `state_values[next_s]`, which records the value for each `next_s` whenever this state is reached from state s with action a.

5.

Return Statement:

6. `state_values` is included in the return statement to provide access to the accumulated values for each state.

Note:

The `transitions` and `rewards` functions are expected to be defined elsewhere and should return the probability distribution over next states and rewards for taking an action in a given state respectively.